KDA Context Parallelism (KCP)

Splitting KDA's recurrence across GPUs with fixed-size messages

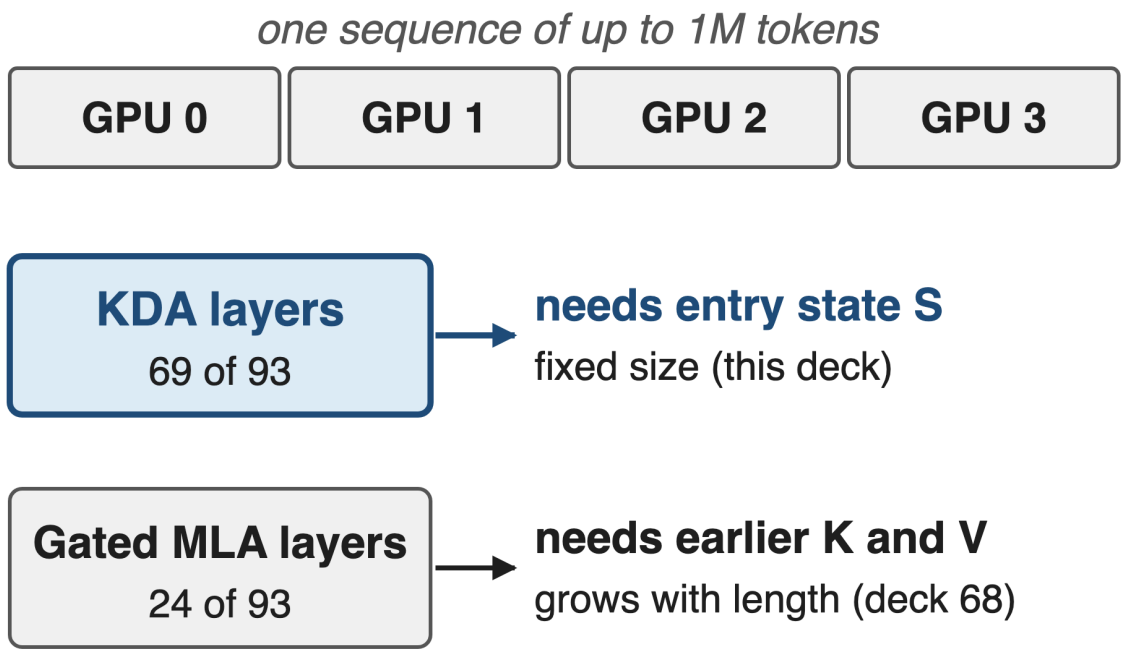
Moonshot AI · Kimi K3 (open implementation: flash-linear-attention PR #691)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Parallelism Strategy · arxiv.org/abs/2607.24653

K3 splits million-token sequences across GPUs

- Context grows 8K → 1M during training
- Pre-training combines PP, EP, ZeRO and CP
- A KDA layer on GPU i needs only its entry state



Head-split CP gets 55% out of four GPUs

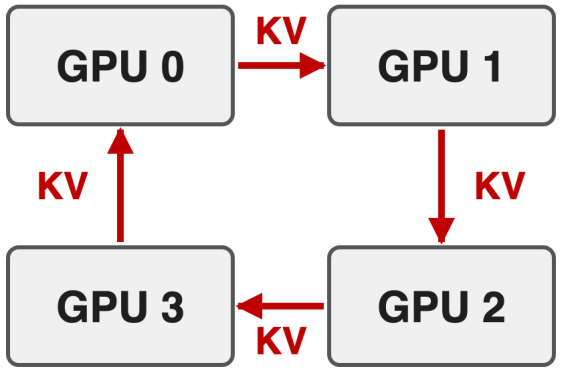
55%

scaling efficiency of head-split (all-to-all) context parallelism for KDA forward + backward on 4 GPUs; 37% for the forward pass alone

flash-linear-attention PR #691: 32K tokens, CP = 4, 64 heads, head dim 128, H800; efficiency = 1-GPU time / (4 × CP time)

Three ways to split a sequence fall short

Softmax-style CP



ships K and V blocks

grows with length

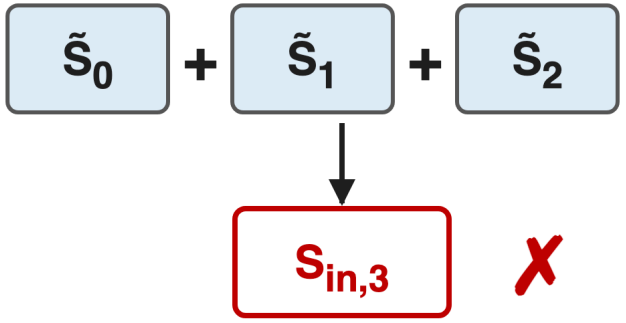
Head-split all-to-all



CP degree ≤ heads

recurrence stays full length

Sum local states



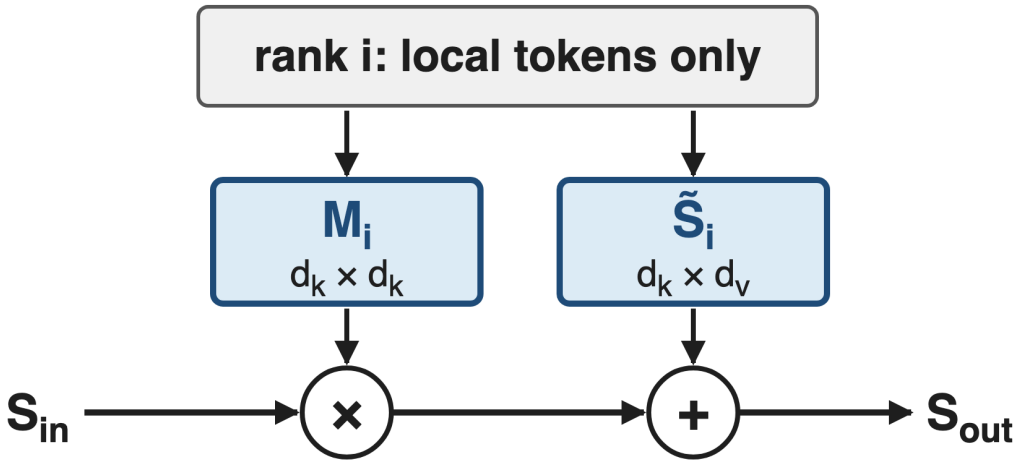
exact only if S just adds up

KDA multiplies S by M

KDA's update: new S = M · old S + β·k·v^T, with M = (I – β·k·k^T)·Diag(α) per token

Insight: a segment is an affine map of S

- M and the zero-start state need only local tokens
- Affine maps compose: a prefix scan

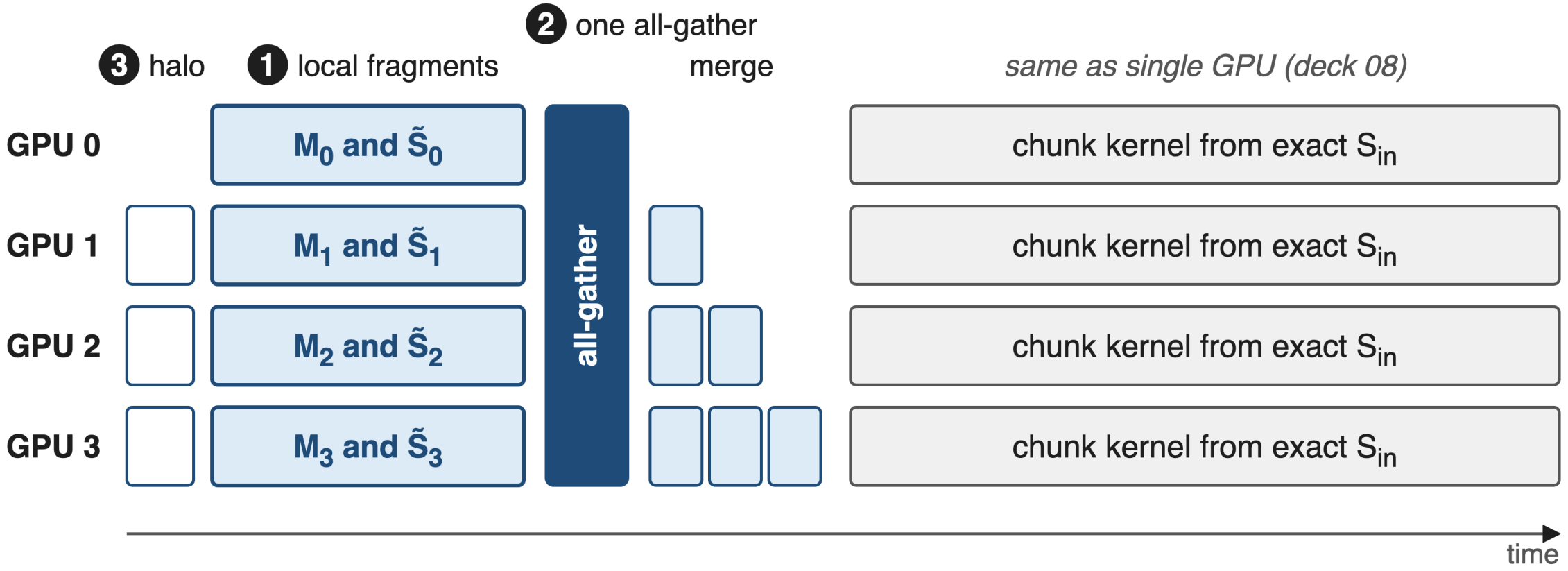


$$S_{out} = M_i \cdot S_{in} + \tilde{S}_i$$

affine maps compose: prefix scan

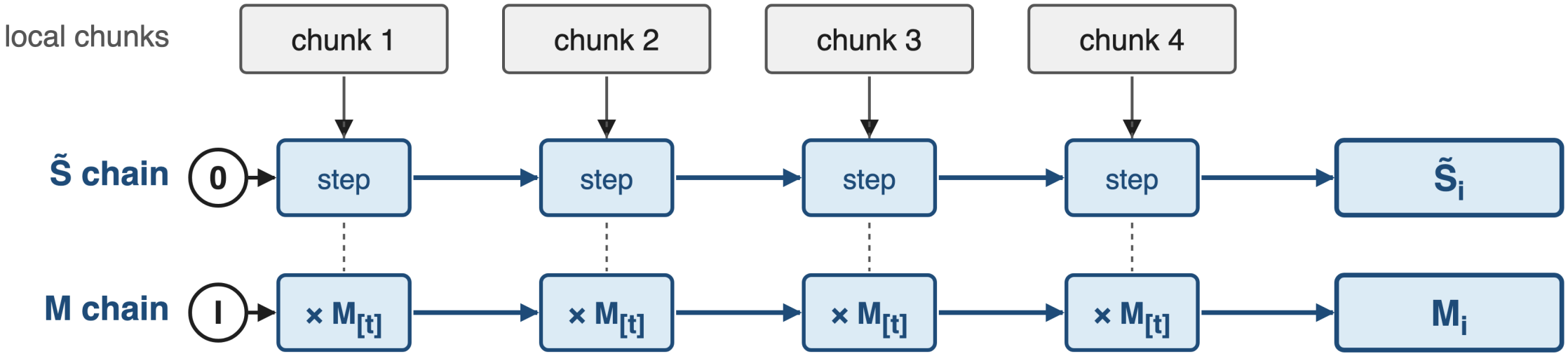
Exchange two fixed-size fragments, never tokens or KV

KCP: local fragments, one all-gather, local merge



❶ local fragments · ❷ one all-gather + in-order merge · ❸ documents and convolution halo (schematic widths)

❶ Each rank builds both fragments on its own

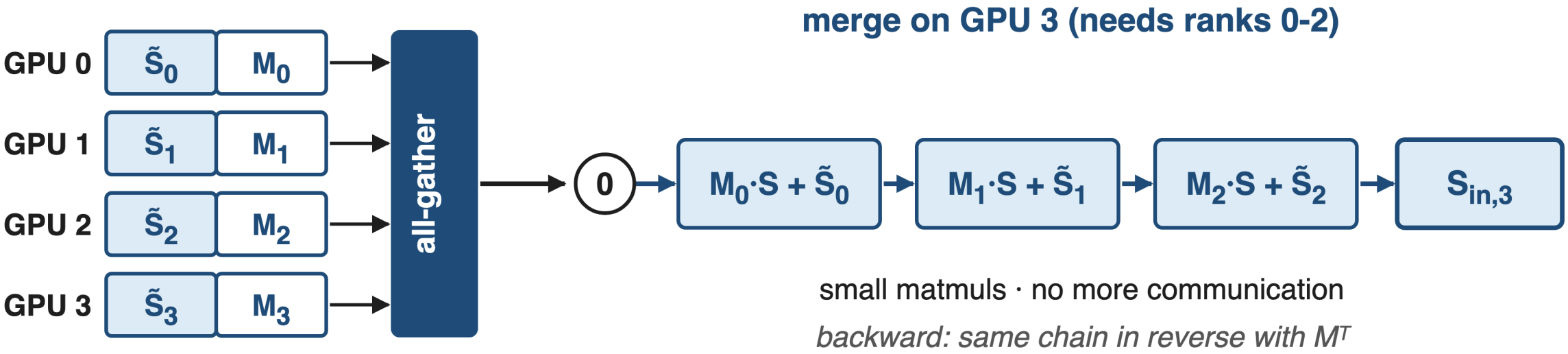


$$\text{step: } \tilde{S} \leftarrow \mathrm{Diag}(\gamma)\cdot\tilde{S} + (\Gamma \odot K)^{T}\cdot(U - W\cdot\tilde{S})$$

$$M_{[t]} = \mathrm{Diag}(\gamma) - (\Gamma \odot K)^{T}\cdot W$$

For any entry state S: exit state = M·S + zero-start state (K3 Eq. 17)

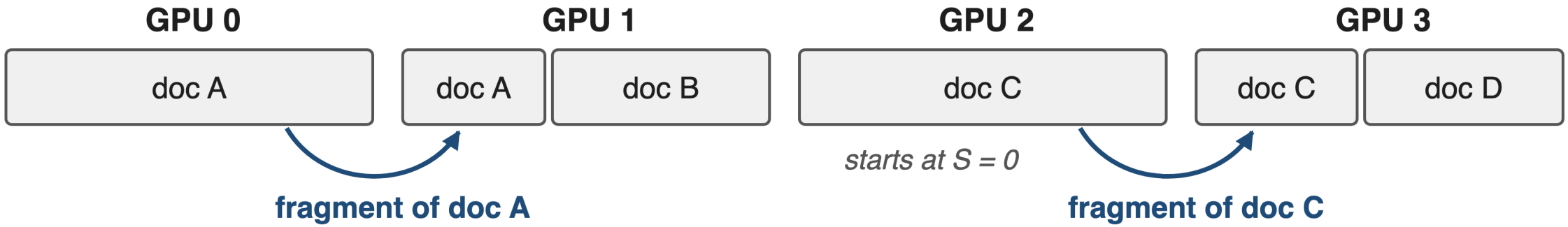
❷ One all-gather, then a local in-order merge



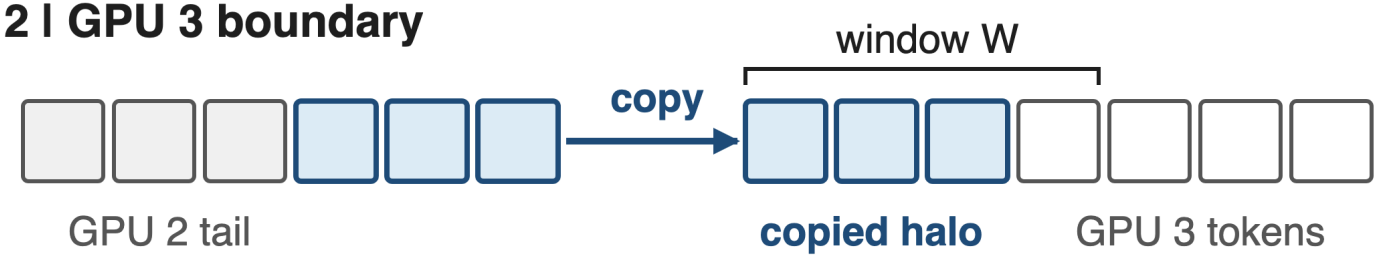
payload per GPU: $H \times d_k \times (d_k + d_v)$ values · no dependence on length

96 heads × head dim 128 in FP32: 12 MiB per GPU per layer, at any length

❸ Packed documents and convolution halos



ShortConv halo at the GPU 2 | GPU 3 boundary

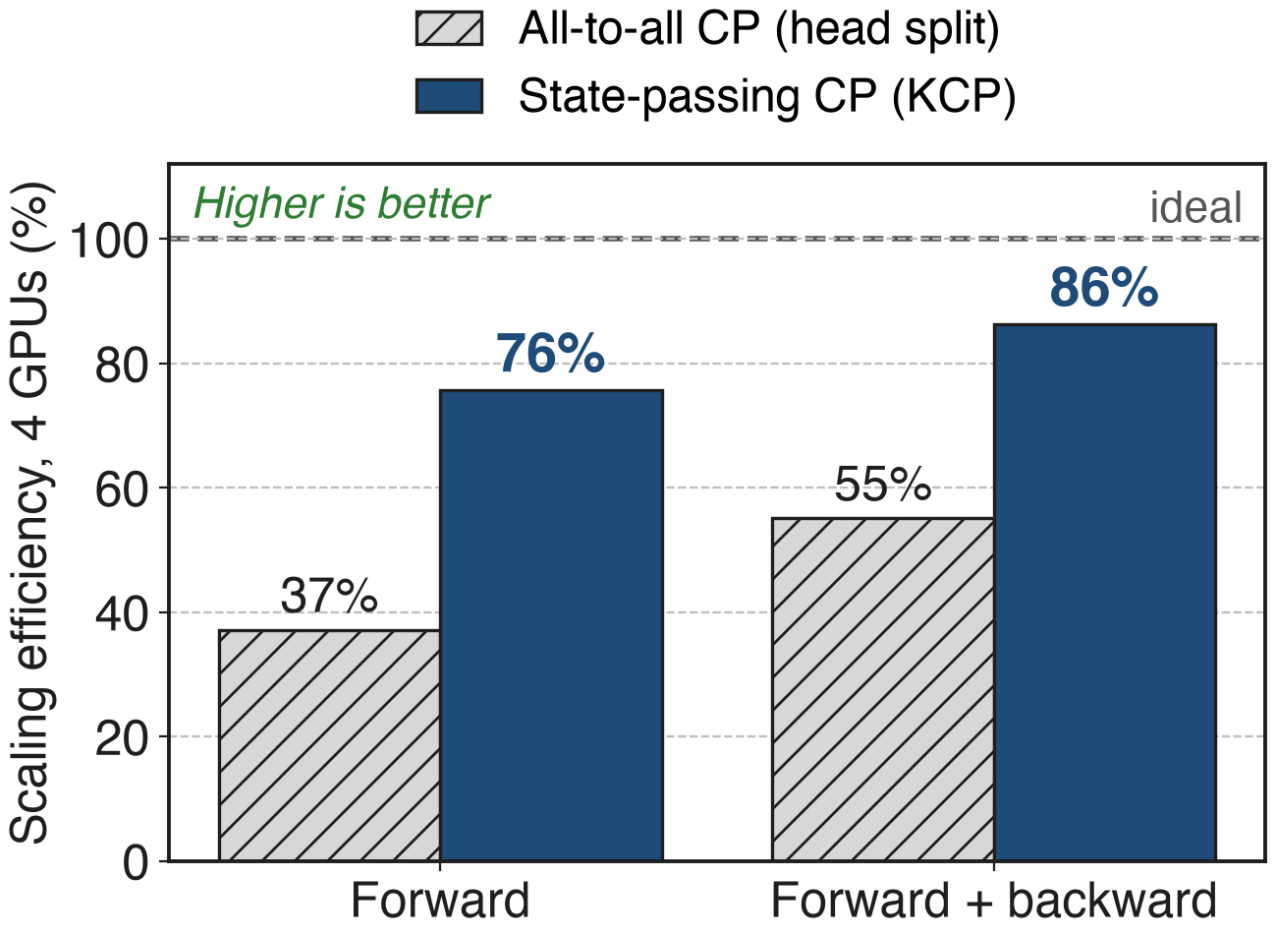


W – 1 tail tokens per boundary · halo gradients return in reverse

Merges never cross documents; only a rank's first sequence can continue

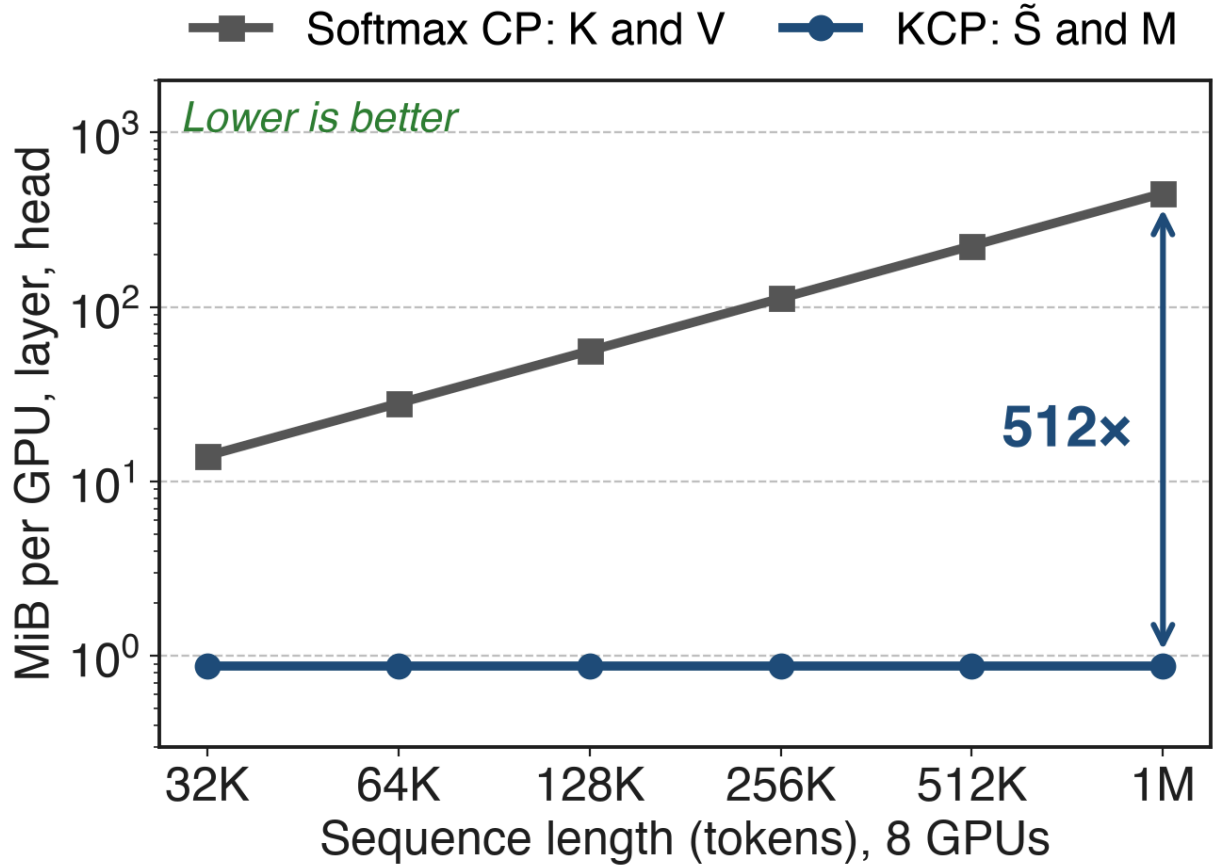
KCP keeps 86% efficiency on 4 GPUs

- 32K tokens, 64 heads, H800 (FLA code)
- Fwd + bwd: 36.2 ms on 1 GPU → **10.5 ms** on 4
- **1.56×** faster than head-split CP



Analytic: 512× less traffic than KV at 1M

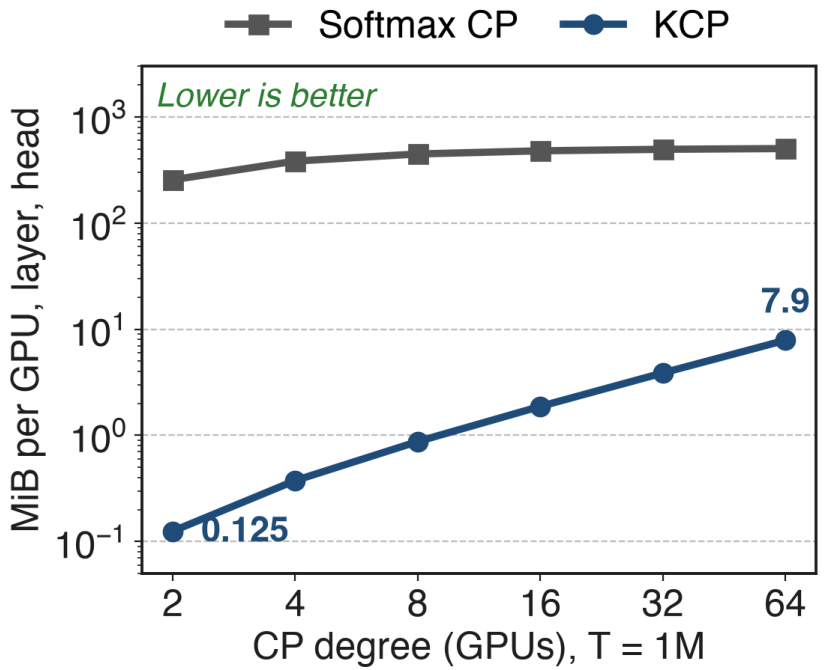
- Softmax CP gathers K and V: grows with length
- KCP gathers two fragments: fixed size
- 1M tokens, 8 GPUs: 448 vs 0.875 MiB per head



analytic; head dim 128, BF16 K and V (MHA-equivalent) vs FP32 fragments

Costs: grows with GPU count, MLA still ships KV

- Payload and merge grow with CP degree
- Extra pre-pass kernels; FP32 chain
- 24 MLA layers still need KV-exchanging CP



analytic, T = 1M, head dim 128

Public timings come from FLA's independent code (32K, 4 GPUs); K3 reports none

Summary

- Head-split CP for KDA: **55%** efficiency on 4 GPUs
- **KCP**: exit = M·entry + zero-start state; gather, merge locally
- **86%** efficiency (fwd + bwd); traffic independent of length
- Chain: KDA deck 05 · FlashKDA deck 08 · softmax CP deck 68

https://arxiv.org/abs/2607.24653